

■ Enterprise Software Development Blueprint: Project EduNexus

****Project Name:** EduNexus (College Student-Teacher Discussion Portal)**

****Version:** 1.0.0**

****Status:** Architecture Design Phase**

****Architect:** Principal Software Architect (Google)**

■ Project Overview

****EduNexus**** is a high-scale, centralized communication and collaboration platform designed for academic institutions. Unlike generic messaging apps, EduNexus is tailored to the pedagogical lifecycle. It facilitates asynchronous academic discussions, resource sharing, and structured Q&A; between students and faculty. The system is designed to handle high concurrency during peak academic hours, ensuring low-latency interactions and strict data privacy compliance (FERPA/GDPR equivalent).

■ Business Objective

* ****Centralize Knowledge:**** Create a single source of truth for course-related queries, reducing redundant email communication by up to 60%.

* ****Enhance Engagement:**** Increase student participation through threaded discussions and real-time notifications.

* ****Data-Driven Insights:**** Provide faculty with analytics on student engagement and common learning bottlenecks.

* ****Scalability:**** Ensure the platform can support multiple departments and thousands of concurrent users across a campus ecosystem.

■ User Roles

1. ****Super Admin:**** Manages institutional settings, department creation, and system-wide audits.

2. ****Faculty (Teachers):**** Create course boards, moderate discussions, pin important announcements, and upload resources.

3. ****Student:**** Join courses, ask questions, reply to threads, and participate in peer-to-peer learning.

4. ****Teaching Assistant (TA):**** Intermediate role with moderation privileges and the ability to endorse "Correct Answers."

■ Functional Requirements

* ****Authentication & Authorization:**** Secure SSO integration (Google Workspace/Active Directory) with Role-Based Access Control (RBAC).

* ****Course-Based Workspaces:**** Automatic or manual creation of discussion "rooms" based on course codes.

* ****Threaded Discussions:**** Support for nested replies, rich-text formatting (Markdown/LaTeX for math), and code snippets.

- * **Real-time Notifications:** In-app, email, and push notifications for mentions and replies.
- * **File Management:** Secure uploading and sharing of PDFs, slide decks, and diagrams.
- * **Search Engine:** Full-text search across all discussions and uploaded documents.
- * **Endorsement System:** Teachers can mark specific replies as "Instructor Verified."
- * **Analytics Dashboard:** Metrics on student activity, most active threads, and response times.

■ Non-Functional Requirements

- * **Availability:** 99.9% Uptime SLA using multi-zone deployment.
- * **Scalability:** Horizontal auto-scaling of microservices based on CPU/Memory utilization.
- * **Security:** AES-256 encryption at rest; TLS 1.3 for data in transit.
- * **Performance:** < 200ms latency for API responses; < 100ms for real-time socket events.
- * **Auditability:** Comprehensive logging of all administrative actions and data access.

■ Suggested Technology Stack

| Layer | Technology |

| :--- | :--- |

| **Frontend** | React.js (Next.js), TypeScript, Tailwind CSS, TanStack Query |

| **Backend** | Node.js (NestJS framework) or Go (Golang) |

| **Database (Primary)** | PostgreSQL (Managed via Google Cloud SQL) |

| **Cache/Real-time** | Redis (Pub/Sub and Session management) |

| **Search** | Elasticsearch or Algolia |

| **Cloud Infrastructure** | Google Cloud Platform (GCP) |

| **DevOps** | Docker, Kubernetes (GKE), GitHub Actions (CI/CD) |

| **Authentication** | Firebase Auth or Auth0 (OIDC/SAML) |

| **File Storage** | Google Cloud Storage (Signed URLs for security) |

■ Suggested Folder Structure

```text

edu-nexus/

■ apps/

■ web/ # Next.js Frontend

■ api/ # NestJS Backend API

■ workers/ # Background jobs (Email, Notifications)

■ packages/

■ common/ # Shared TS interfaces/DTOs

■ database/ # Prisma/TypeORM schemas and migrations

■ ■ ■ ■ ui-kit/ # Shared Tailwind components

■ ■ ■ infrastructure/

■ ■ ■ ■ terraform/ # IaC for GCP resources

■ ■ ■ ■ k8s/ # Kubernetes manifests

■ ■ ■ README.md

...

---

## # ■ REST API Endpoints

| Method | Endpoint | Purpose |

| :--- | :--- | :--- |

| `POST` | `/api/v1/auth/login` | User authentication and JWT issuance |

| `GET` | `/api/v1/courses` | List all courses for the authenticated user |

| `GET` | `/api/v1/courses/:id/threads` | Fetch discussion threads for a specific course |

| `POST` | `/api/v1/threads` | Create a new discussion thread |

| `POST` | `/api/v1/threads/:id/comments` | Post a reply to a thread |

| `PATCH` | `/api/v1/comments/:id/verify` | (Faculty only) Mark an answer as verified |

| `GET` | `/api/v1/search?q=...` | Global search across threads and files |

| `POST` | `/api/v1/upload/resource` | Upload course material (returns Cloud Storage URL) |

---

## # ■ Database Design

### \*\*Users Table\*\*

- `id` (UUID, PK)

- `email` (String, Unique)

- `role` (Enum: STUDENT, TEACHER, ADMIN)

- `department\_id` (FK)

### \*\*Courses Table\*\*

- `id` (UUID, PK)

- `course\_code` (String)

- `name` (String)

- `instructor\_id` (FK to Users)

### \*\*Threads Table\*\*

- `id` (UUID, PK)

- `course\_id` (FK to Courses)

- `author\_id` (FK to Users)

- `title` (String)

- `content` (Text/Markdown)

- `is\_pinned` (Boolean)

### **Comments Table**

- `id` (UUID, PK)

- `thread\_id` (FK to Threads)

- `author\_id` (FK to Users)

- `parent\_id` (FK for nested replies)

- `content` (Text)

- `is\_verified` (Boolean)

---

## # ■ Deployment Architecture

The system will follow a **Cloud-Native Microservices** pattern:

1. **Traffic Entry:** Cloud Load Balancer distributes traffic to GKE clusters.
2. **API Gateway:** Handles rate limiting, SSL termination, and request routing.
3. **Service Layer:** Containerized Node.js services managed by Kubernetes.
4. **State Management:**
  - \* **PostgreSQL** for relational, transactional data.
  - \* **Redis** for high-speed socket connections and caching.
5. **Observability:** Google Cloud Operations Suite (formerly Stackdriver) for logging, monitoring, and distributed tracing.

---

## # ■ Development Roadmap

### **Week 1: Foundation & Authentication**

- \* Initialize Monorepo.
- \* Setup CI/CD pipelines and GCP environments (Dev/Staging).
- \* Implement SSO/Auth modules and User RBAC.

### **Week 2: Core Course & Thread Logic**

- \* Develop Course Management API.
- \* Implement Thread creation and CRUD operations.
- \* Build the Frontend dashboard and course listing views.

### **Week 3: Real-time Interaction & Files**

- \* Integrate Socket.io/Redis for real-time comment updates.
- \* Implement File upload service with GCP Bucket integration.
- \* Build the Markdown editor for the Frontend.

### **Week 4: Search & Analytics**

- \* Setup Elasticsearch indexing for threads.

- \* Implement Advanced Search UI.
- \* Develop Faculty Analytics Dashboard (charts for engagement).

#### ### \*\*Week 5: Quality Assurance & Security\*\*

- \* Conduct Penetration Testing.
- \* Perform Load Testing (targeting 5k concurrent users).
- \* Execute UAT (User Acceptance Testing) with a small group of students.

#### ### \*\*Week 6: Deployment & Handover\*\*

- \* Production environment hardening.
- \* Data migration/seeding.
- \* Official launch and post-launch monitoring.

---

\*End of Blueprint\*